UNIVERSITÉ SORBONNE PARIS NORD

SAE Bases de Données et langage SQL

Filière : Informatique

Projet SAE Bases de Données et langage SQL

Saé104

Rédigé par :

- LAKSHMINARAYANAN Loïc

Encadré par :
Mme. Zargayouna

SOMMAIRE

Sommaire :

Sommaire :..... 1
I. Script manuel de création de la base de données..... 2
II. Modélisation et script de création « avec AGL »..... 3
Illustrations comparatives cours/AGL commentée d'une association fonctionnelle :..... 3
Illustrations comparatives cours/AGL commentée d'une association maillée :..... 3
Modèle physique de données réalisé avec l'AGL :..... 5
Script SQL de création des tables généré automatiquement par l'AGL :..... 5
Les différences entre les scripts produits manuellement et automatiquement :..... 8
III. Peuplement des tables..... 9
Script de peuplement de la base de données :..... 9
IV. Annexe :..... 11

I. Script manuel de création de la base de données

Voici le code pour la création des tables :

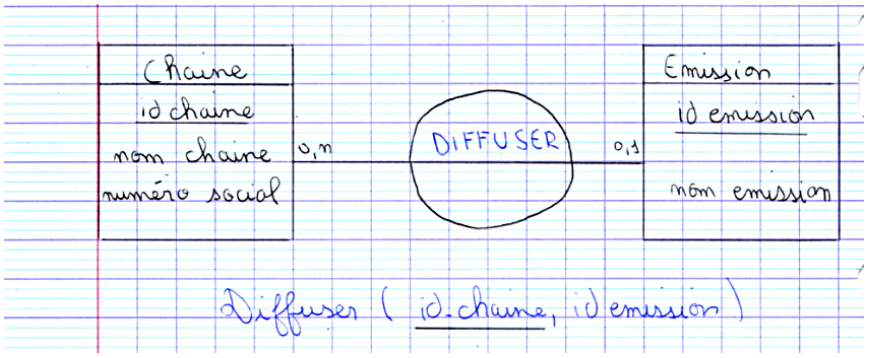
```
CREATE TABLE regions (
        code_region INTEGER PRIMARY KEY,
        nom_region VARCHAR NOT NULL);


CREATE TABLE departements (
        code_departement VARCHAR PRIMARY KEY,
        code_region INTEGER REFERENCES regions(code_region),
        nom_département VARCHAR NOT NULL);


CREATE TABLE communes (
        code_commune VARCHAR PRIMARY KEY,
        code_departement VARCHAR REFERENCES departements(code_departement),
        nom VARCHAR NOT NULL);


CREATE TABLE sites(
        idSite VARCHAR PRIMARY KEY,
        nom_site VARCHAR NOT NULL,
        code_commune VARCHAR REFERENCES communes(code_commune),
        dateDeclaration DATE,
        typeEau VARCHAR,
        longitude REAL,
        latitude REAL);


CREATE TABLE evenements(
        idEvenement INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
        idSite VARCHAR REFERENCES sites(idSite),
        evenement VARCHAR,
        debut DATE,
        fin DATE,
        mesure VARCHAR);


CREATE TABLE analyses(
        idAnalyse INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
        idSite VARCHAR REFERENCES sites(idSite),
        datePrelevement DATE NOT NULL,
        enterocoques INTEGER,
        escherichia INTEGER);
```

Le type "GENERATED ALWAYS AS IDENTITY PRIMARY KEY" est demandé dans le peuplement des tables.

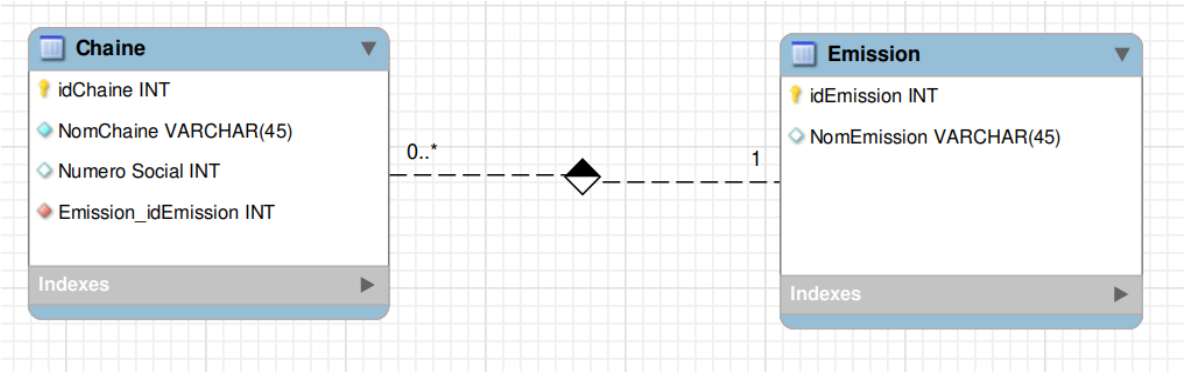
II. Modélisation et script de création « avec AGL »

Pour cette partie, j'ai opté pour l'AGL, **MYSQL WORKBENCH**. Pour montrer en quoi le modèle physique proposé par l'AGL diffère du modèle conceptuel vu en cours. J'ai pris cet exemple d'un exercice du TD.

Illustrations comparatives cours/AGL commentée d'une **association fonctionnelle** :

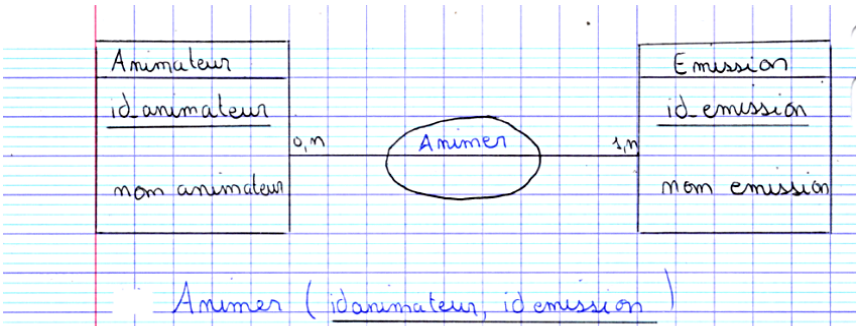


Comme nous pouvons le constater dans le cours, une **association fonctionnelle** est représentée par un losange reliant deux entités, avec des cardinalités dont l'un des côtés est de type (1,1). En effet, dans cet exemple l'association "DIFFUSER" relie les entités "Chaîne" et "Émission", une chaîne peut diffuser plusieurs émissions (0,n), mais une émission n'est diffusée que par une chaîne(1,1).

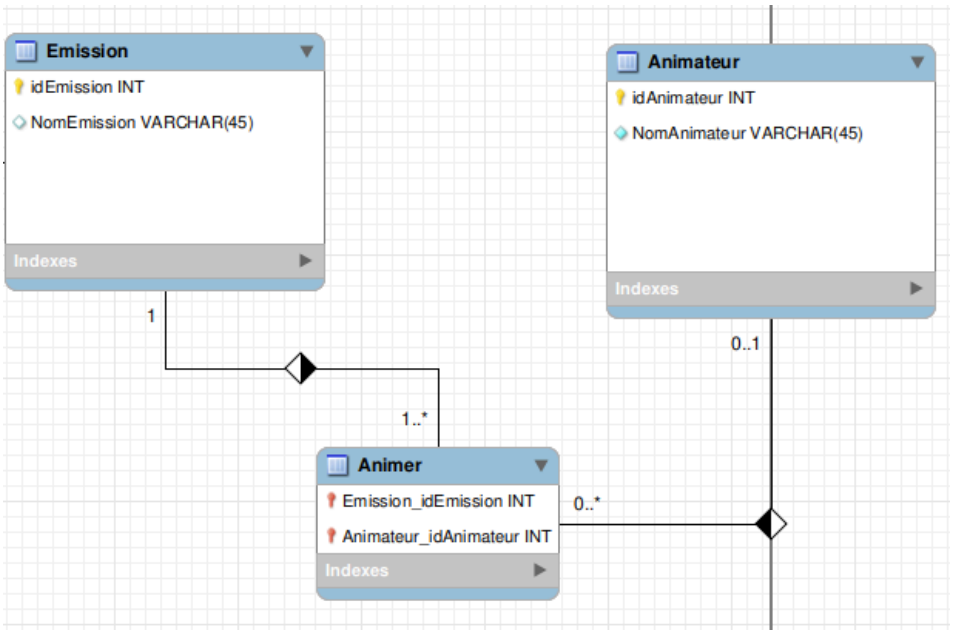


Après le passage au modèle physique, j'ai changé l'affichage de la notation des relations, j'ai mis le modèle "**classic"** car par défaut les différentes relations était en "**crow's foot**" ce qui rendait la compréhension des tables plus complexes. Après tous ses réglages, on remarque que l'association "DIFFUSER" est remplacée par l'ajout d'une clé étrangère dans la table "Chaîne" car chaque émission ne peut être associée qu'à une seule chaîne. En effet, la clé primaire de la table "Émission" est ajoutée comme clé étrangère dans la table "Chaîne". De plus, nous pouvons également voir que les clés primaires ne sont pas soulignées à la place ils sont précédés d'une ampoule (pour les deux associations maillées et fonctionnelles).

Illustrations comparatives cours/AGL commentée d'une **association maillée** :

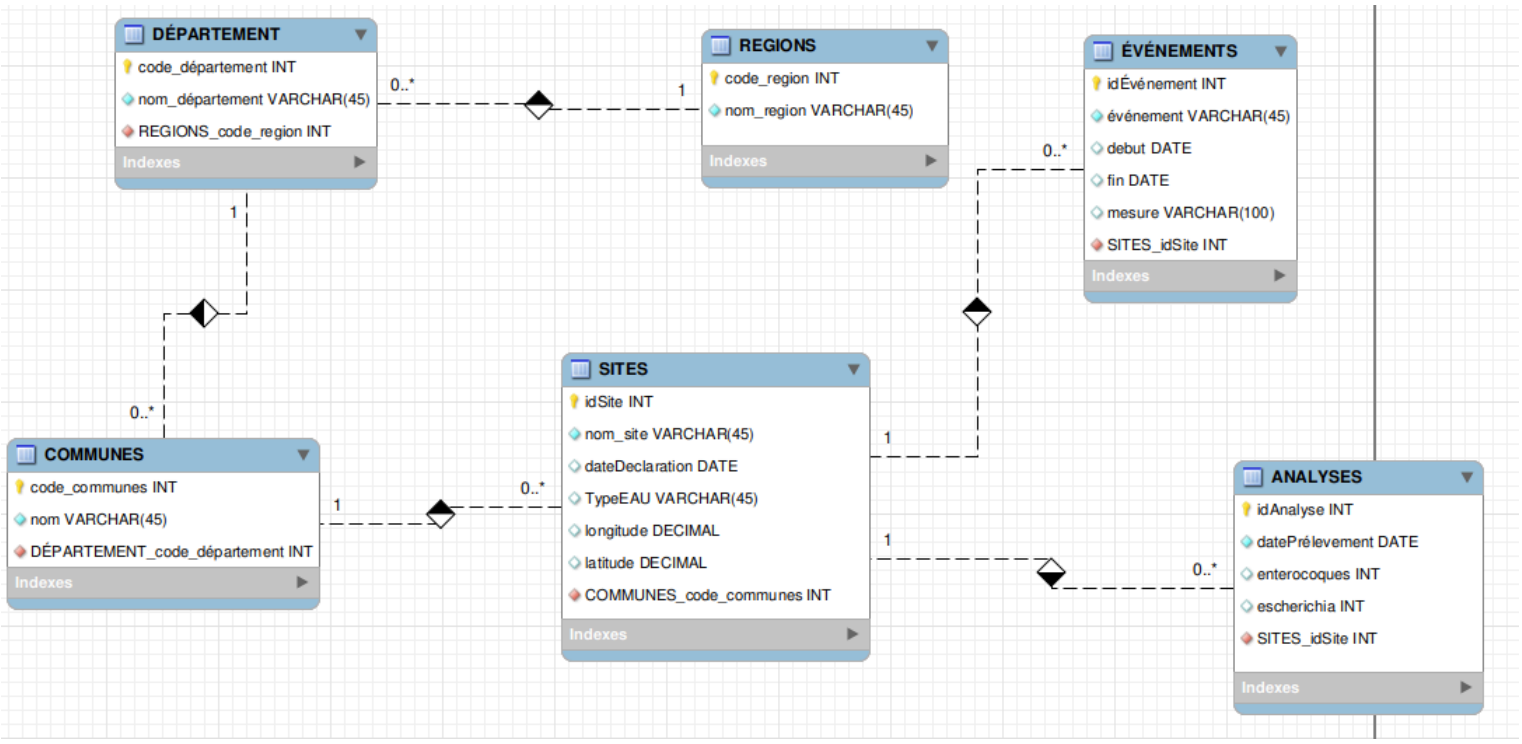


Comme nous pouvons le constater dans le cours, une **association maillée** est représentée par un losange reliant deux entités, avec des cardinalités de type ((0,n) et (1,n)) des deux côtés. Dans cet exemple, l'association "ANIMER" relie les entités "Animateur" et "Émission" , un animateur peut animer plusieurs émissions et une émission peut être animée par plusieurs animateurs.



Après le passage au modèle physique, on remarque que l'association "Animer" s'est transformée en une table contenant les clés primaires des deux entités "Animateur" et "Émission". Ces deux clés étrangères forment donc la clé primaire de cette nouvelle table.

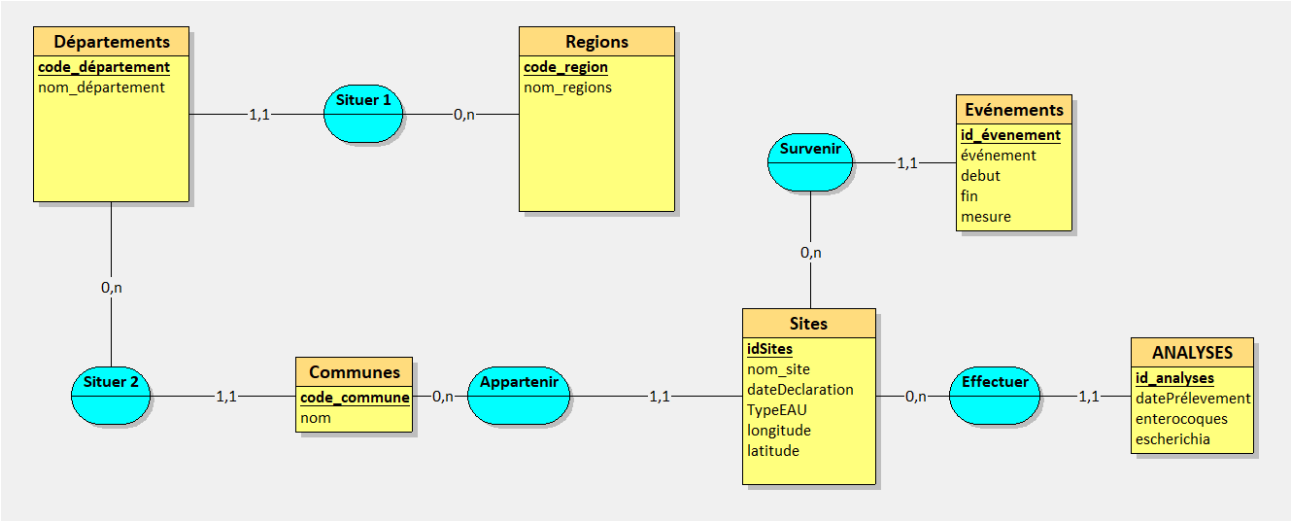
Modèle physique de données réalisé avec l'AGL :



Script SQL de création des tables généré automatiquement par l'AGL :

Lors de la réalisation de cette partie, j'ai rencontré plusieurs problèmes liés au choix de l'AGL. Comme on peut le voir sur les images d'avant j'utilisais **MySql WorkBench** pour produire le modèle physique et générer automatiquement le script de création. Mais, après plusieurs essais et des recherches, je me suis rendu compte que le script généré était spécifique à MySQL et qu'il n'était pas directement compatible avec PostgreSQL (le SGBD demandé).

Je me suis ensuite renseigné auprès de mes camarades ayant eu le même problème (impossibilité de générer directement un script PostgreSQL avec MySQL Workbench), c'est alors que mes camarades m'ont conseillé une alternative étant **Looping**. C'est pour cette raison que je me suis orienté vers **Looping** et voici le modèle physique de données réalisé avec cette AGL :



Voici le script obtenu :

```
CREATE TABLE Regions(
        code_region INT,
        nom_regions VARCHAR(50) NOT NULL,
        PRIMARY KEY(code_region)
);

CREATE TABLE Départements(
        code_département VARCHAR(50),
        nom_département VARCHAR(50) NOT NULL,
        code_region INT NOT NULL,
        PRIMARY KEY(code_département),
        FOREIGN KEY(code_region) REFERENCES Regions(code_region)

);


CREATE TABLE Communes(
       code_commune VARCHAR(50),
       nom VARCHAR(50) NOT NULL,
       code_département VARCHAR(50) NOT NULL,
       PRIMARY KEY(code_commune),
       FOREIGN KEY(code_département) REFERENCES Départements(code_département)

);

CREATE TABLE Sites(
        idSite VARCHAR(50),
        nom_site VARCHAR(50) NOT NULL,
        dateDeclaration DATE,
        TypeEAU VARCHAR(50),
        longitude DECIMAL(15,2),
        latitude DECIMAL(15,2),
        code_commune VARCHAR(50) NOT NULL,
        PRIMARY KEY(idSites),
        FOREIGN KEY(code_commune) REFERENCES Communes(code_commune)
);


CREATE TABLE Evénements(
        id_évenement INT,
        événement VARCHAR(50) NOT NULL,
        debut DATE,
        fin DATE,
        mesure VARCHAR(100),
        idSite VARCHAR(50) NOT NULL,
```

```
	PRIMARY KEY(id_évenement),
	FOREIGN KEY(idSites) REFERENCES Sites(idSites)
);


CREATE TABLE ANALYSES(
	id_analyses INT,
	datePrélevement DATE NOT NULL,
	enterocoques INT,
	escherichia INT,
	idSite VARCHAR(50) NOT NULL,
	PRIMARY KEY(id_analyses),
	FOREIGN KEY(idSites) REFERENCES Sites(idSites)
);
```

Les différences entre les scripts produits manuellement et automatiquement :

Ces deux scripts (écrit manuellement et le script généré automatiquement par Looping) arrivent tous les deux à la même finalité, une structure de base de données, les tables sont créées avec leurs clés primaires et les tables sont reliées par des clés étrangères. Malgré tout, des différences sont visibles entre ses deux approches.

Tout d'abord, on peut remarquer que mon script manuel est plus court, tandis que le script généré par l'AGL est plus lisible. En effet, les noms utilisés sont plus cohérents mais il ajoute automatiquement certaines contraintes, comme certaines conventions de nommage.

De plus, l'AGL impose des choix techniques comme par exemple, les types sont définis par défaut (INT, VARCHAR(45)...), c'est pourquoi certains choix de types diffèrent entre les deux scripts comme celui de la "Longitude" et "Latitude", dans le script manuel, leur type est un "REAL" alors que dans le script généré, leur type est un "DECIMAL(15,2)".

Ensuite, on peut remarquer que l'AGL a ajouté plus de contraintes NOT NULL, code_region INT NOT NULL dans Départements, code_département INT NOT NULL dans Communes ou encore code_commune VARCHAR(50) NOT NULL dans Sites.

La plus grande différence sont les relations avec les tables. En effet, Looping déclare les clés étrangères explicitement en fin de table :
FOREIGN KEY(idSites) REFERENCES Sites(idSites)
tandis que dans mon script, j'utilise une écriture plus directe dans l'attribut :
region INTEGER REFERENCES regions(code).

Finalement, Looping impose des tailles à mettre (VARCHAR(50)) alors que dans mon script manuel, la taille n'est pas précisée (pas d'obligation).

Pour conclure, malgré que mon script soit plus simple à comprendre, le script généré par l'AGL reste meilleur grâce à ces nombreuses contraintes.

III. Peuplement des tables

Script de peuplement de la base de données :

```
DROP TABLE IF EXISTS temp_depart_region, temp_sites, temp_evenements, temp_analyses;

--------------------- CRÉATION DES TABLES TEMPORAIRES ---------------------

CREATE TABLE temp_depart_region (
        code_departement VARCHAR,
        nom_departement  VARCHAR,
        code_region      INTEGER,
        nom_region       VARCHAR
);
--------------------------- COPIE DES FICHIERS ---------------------------

\copy temp_depart_region FROM '/home/loicl/Downloads/departements-france.csv'  CSV HEADER ;

\copy temp_sites FROM
'/home/loicl/Downloads/liste-des-sites-de-baignade-saison-balneaire-2024.csv' CSV HEADER
DELIMITER ';' ENCODING 'LATIN1';

--------------------------- AJOUT DES DONNÉES DANS LES TABLES  ----------------------------

INSERT INTO regions SELECT DISTINCT code_region, nom_region
FROM temp_depart_region;

INSERT INTO departements
SELECT DISTINCT
        LTRIM(code_departement,'0'),
        code_region,
        nom_departement
FROM temp_depart_region;

INSERT INTO communes SELECT DISTINCT code_insee, LTRIM(departement, '0'),
nom_commune
 FROM temp_sites;

INSERT INTO sites (idSite, nom_site, code_commune, dateDeclaration, typeEau, longitude, latitude)
SELECT DISTINCT idsite, nom_site, code_insee, date_declaration, type_eau,
CAST(REPLACE(longitude, ',' ,  '.') AS REAL), CAST(REPLACE(latitude,  ',' ,  '.') AS REAL)
FROM temp_sites;

INSERT INTO evenements (idSite, evenement, debut, fin, mesure)
SELECT idsite, type_evenement, debut, fin, mesures_gestion
FROM temp_evenements;

INSERT INTO analyses(idSite, datePrelevement, enterocoques, escherichia)
SELECT idsite, date_prelevement, enterocoques, escherichia
FROM temp_analyses;
```

Description commentée des différentes étapes de votre script de peuplement :

Dans un premier temps, je supprime les tables temporaires si elles existent déjà avec :
"DROP TABLE IF EXISTS temp_depart_region, temp_sites, temp_evenements, temp_analyses; ",
Ce qui va me permettre de relancer le script plusieurs fois sans avoir de problèmes.

Puis, je crée les quatres temporaires temp_depart_region, temp_sites, temp_evenements et temp_analyses (dans lesquelles, je vais importer les fichiers CSV, puis transférer les données vers les tables finale avec des INSERT).
Étant donné que j'utilise ce même principe pour les quatres tables, je ne les ai pas répété à chaque fois car la logique est la même. Mais il y a un cas particulier, la table temp_analyse. En effet, le fichier CSV des analyses contient trois colonnes vides supplémentaires qui sont présentes dans le fichier mais ne sont pas utilisées par la base. Pour éviter une erreur au moment de l'import, j'ai rajouté trois attribut "vide" (code mit en annexe).

Les fichiers CSV sont ensuite importés dans les tables temporaires avec le \copy, mais pour que l'import fonctionne correctement j'ai du utiliser certaines options (conseillé par Mr. Laurent Audibert sur Discord) tel que "CSV HEADER", qui va indiquer que la première ligne du fichier correspond au noms des colonnes et ne doit donc pas être interprétée comme étant des données.
Ensuite l'option "DELIMITER ';' ", qui va découper les lignes du fichiers selon le séparateur ";" car les fichiers fournis utilisent tous des points-virgules comme des séparateurs de champs et non pas une virgule.
Et finalement, l'option ENCODING 'LATIN1' est utilisée parce que certains fichiers contiennent des caractères accentués (noms des régions, de département ou encore de communes) et ne sont pas encodés en UTF-8.
Les commandes \copy pour les tables temp_evenements et temp_analyses, ne sont pas mises dans le code "simplifié" (ils sont dans le code final), car elles reposent sur le même principe que celle de temp_sites (même délimiteur (;) et même encodage ('LATIN1')).

Et dernièrement, on va insérer les données dans les tables finales à partir des tables temporaires grâce à des requêtes de projection : INSERT INTO et SELECT. J'utilise notamment l'option DISTINCT pour éviter les doublons parce que dans les CSV certaines informations apparaissent plusieurs fois (comme une région qui se répète pour chaque département).
Pour le peuplement de "departements", la fonction LTRIM est utilisée pour retirer les zéros en début (par exemple 005 devient 5) ce qui va rendre le format plus simple et va nous éviter des erreurs (dans le peuplement de "communes", il est utilisé pour la même raison).
Pour le peuplement de "sites", les attributs "Longitude" et "Latitude" doivent être convertis en REAL, je ne l'ai pas directement mit lors de la création de la table temporaire car il faut d'abord remplacer la virgule décimale par un point avec la fonction REPLACE (par exemple 5,8210 devient 5.8210) car PostgreSQL n'interprète pas cela comme des virgules décimales. Puis, j'ai utilisé la fonction CAST(.... AS REAL) pour la conversion.
Enfin, pour le peuplement d' "evenements" et d' "analyse", je n'ai pas eu de grand changement à faire car malgré que l' "idEvenement" et l' "idanalyse" ne sont pas fourni, ils sont générés automatiquement grâce à "GENERATED ALWAYS AS IDENTITY" que j'ai rajouté au script 2.1.

IV. Annexe :

Voici le code complet :

```
DROP TABLE IF EXISTS temp_depart_region, temp_sites, temp_evenements, temp_analyses;

CREATE TABLE temp_depart_region (
        code_departement VARCHAR,
        nom_departement  VARCHAR,
        code_region      INTEGER,
        nom_region       VARCHAR
);

CREATE TABLE temp_sites (
        saison_balneaire VARCHAR,
        region_nom    VARCHAR,
        departement  VARCHAR,
        idsite   VARCHAR,
        precedent_idsite VARCHAR,
        evolution  VARCHAR,
        nom_site    VARCHAR,
        code_insee   VARCHAR,
        nom_commune  VARCHAR,
        date_declaration DATE,
        type_eau  VARCHAR,
        longitude  VARCHAR,
        latitude  VARCHAR
);

CREATE TABLE temp_evenements (
        saison_balneaire VARCHAR,
        region_nom   VARCHAR,
        departement   VARCHAR,
        idsite  VARCHAR,
        type_evenement  VARCHAR,
        debut  DATE,
        fin  DATE,
        mesures_gestion  VARCHAR
);

CREATE TABLE temp_analyses (
        saison_balneaire VARCHAR,
        region_nom  VARCHAR,
        departement   VARCHAR,
        idsite  VARCHAR,
        date_prelevement DATE,
        enterocoques   INTEGER,
        escherichia  INTEGER,
        statut_prelev  VARCHAR,
```

```
	vide1 VARCHAR,
	vide2 VARCHAR,
	vide3 VARCHAR
	);

\copy temp_depart_region FROM '/home/loicl/Downloads/departements-france.csv'  CSV HEADER ;

\copy temp_sites FROM
'/home/loicl/Downloads/liste-des-sites-de-baignade-saison-balneaire-2024.csv' CSV HEADER
DELIMITER ';' ENCODING 'LATIN1';

\copy temp_evenements FROM
'/home/loicl/Downloads/saison-balneaire-2024-informations-sur-la-saison.csv' CSV HEADER
DELIMITER ';' ENCODING 'LATIN1';

\copy temp_analyses FROM '/home/loicl/Downloads/saison-balneaire-2024-resultats-danalyses.csv'
CSV HEADER DELIMITER ';' ENCODING 'LATIN1';

INSERT INTO regions
SELECT DISTINCT
	code_region,
	nom_region
FROM temp_depart_region;

INSERT INTO departements
SELECT DISTINCT
	LTRIM(code_departement,'0'),
	code_region,
	nom_departement
FROM temp_depart_region;

INSERT INTO communes
	SELECT DISTINCT
	code_insee,
	LTRIM(departement, '0'),
	nom_commune
FROM temp_sites;

INSERT INTO sites (idSite, nom_site, code_commune, dateDeclaration, typeEau, longitude, latitude)
SELECT DISTINCT
	idsite,
	nom_site,
	code_insee,
	date_declaration,
	type_eau,
	CAST(REPLACE(longitude, ',' ,  '.') AS REAL),
	CAST(REPLACE(latitude,  ',' ,  '.') AS REAL)
FROM temp_sites;
```

```
INSERT INTO evenements (idSite, evenement, debut, fin, mesure)
SELECT
	idsite,
	type_evenement,
	debut,
	fin,
	mesures_gestion
FROM temp_evenements;

INSERT INTO analyses(idSite, datePrelevement, enterocoques, escherichia)
SELECT
	idsite,
	date_prelevement,
	enterocoques,
	escherichia
FROM temp_analyses;
```